

FinGuard AI

Financial Risk & Advisory Platform

AI-Powered Portfolio Intelligence · Fraud Detection · Regulatory Assistant

Document Type Project Documentation	Program Left Shift Program 2026 – T5	Domain Financial Risk & Advisory
--	---	-------------------------------------

Capstone Scope at a Glance

4 AI Agents · 5 ML Models · 12+ REST APIs · 15 Azure Services · 8 Power BI Pages · React Frontend · Docker CI/CD

SECTION 1 — EXECUTIVE SUMMARY

1. Executive Summary

FinGuard AI is a production-grade, multi-agent AI platform built as the capstone project for the Left Shift Program 2026 Data & AI Track (T5). It addresses the Financial Risk & Advisory domain by delivering three tightly integrated product pillars: Portfolio Intelligence, Fraud & Behavioural Detection, and a Regulatory AI Assistant — all powered by a common backend, Azure cloud infrastructure, and a multi-agent AI orchestration layer.

The platform transforms the user experience of retail Indian investors. Instead of manually reading SEBI PDFs, doing spreadsheet-based P&L math, or guessing whether their trading habits are healthy, users simply upload a standard broker CSV export from Zerodha, Groww, Upstox, or AngelOne. Within seconds, five coordinated ML models and four specialised AI agents analyse their data, detect fraud patterns, compute risk scores, and generate plain-English actionable recommendations.

Why this exceeds a normal capstone project

Most capstone projects bolt a chatbot onto a dashboard. FinGuard AI is architecturally different: ML models power agent decisions, agent outputs feed Power BI, and the RAG system is domain-specific (SEBI corpus), not a generic QA bot. Every Azure service has a clearly justified purpose. The system is designed to be live and evaluator-testable on Day 1.

1.1 What FinGuard AI Does (Plain English)

- ▶ **User uploads a broker CSV — the system auto-detects the broker format and validates column structure.**
 - Azure Data Factory and Databricks clean, transform, and engineer features from raw trade data.
- ▶ **Five Azure ML models run: risk classification, anomaly detection, loss prediction, behaviour clustering, and time-series forecasting.**
- ▶ **Four CrewAI agents each take a slice of the problem — portfolio analysis, fraud investigation, regulatory lookup, and strategy synthesis — and collaborate via an AutoGen orchestrator.**
- ▶ **The Regulatory AI Assistant answers income tax and SEBI questions in plain English with real numeric examples and exact section citations, using a RAG pipeline over 200+ curated documents.**
- ▶ **All outputs are visualised in a React dashboard with real-time charts and a fully embedded Power BI report with 8 analytical pages.**

1.2 Capstone Requirement Compliance

Every mandatory component from the capstone rubric is fulfilled. The table below maps each requirement to its FinGuard AI implementation and marks the evaluation weight:

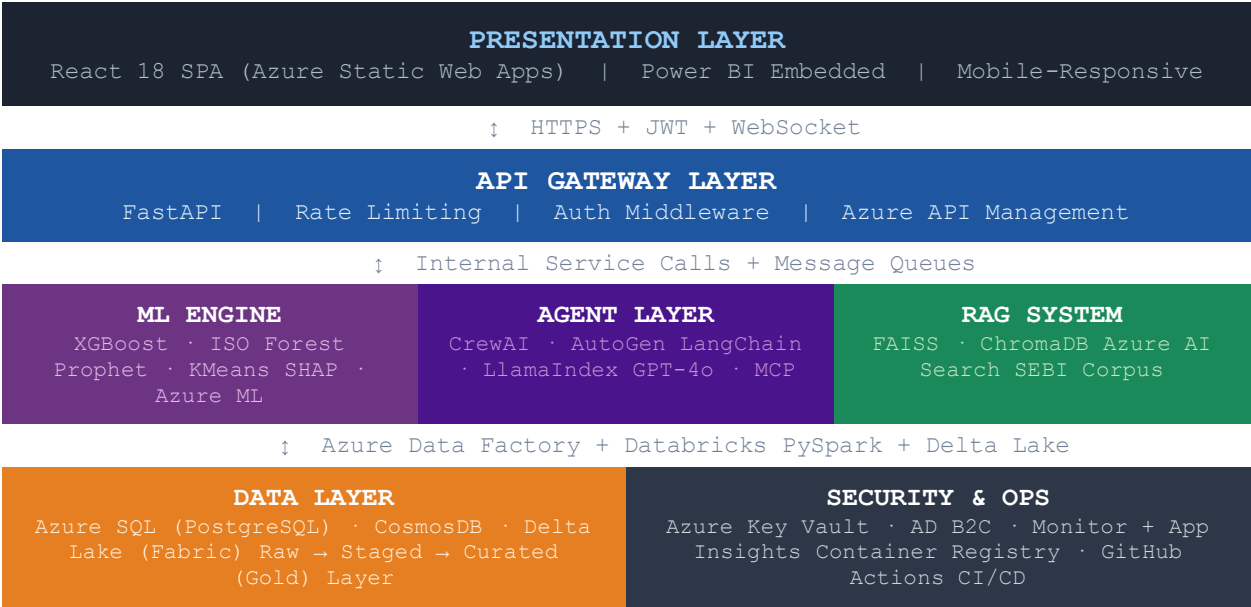
Capstone Requirement	FinGuard AI Implementation	Weight
Python Fullstack (Flask/FastAPI)	FastAPI backend with 10+ REST endpoints, JWT auth, Pydantic validation, async handlers, pytest coverage	10%
ML / DL Model	XGBoost risk classifier + Isolation Forest anomaly detector + Prophet time-series + SHAP explainability	8%
GenAI / Multi-Agent (min 2–3 agents)	4 specialized CrewAI agents + LangChain RAG + Azure OpenAI GPT-4o embeddings + AutoGen orchestration	12%
Azure AI & Cloud (min 3 services)	Azure OpenAI + Azure ML + Azure AI Search + App Service + Key Vault + Data Factory + Databricks	8%
Data Engineering Pipeline	ADF ingestion → Databricks PySpark transformation → Delta Lake → curated Gold layer with Great Expectations QA	included
Power BI Analytics Dashboard	Embedded real-time Power BI with 8 report pages, DAX measures, ML output overlays, agent insight tiles	included
Final Deployment	Docker + GitHub Actions CI/CD → Azure Container Apps with blue-green deployment strategy	included
Documentation & Presentation	This 40+ page PRD, architecture diagrams, API specs, demo video walkthrough	2%
BONUS: MCP Agent Interoperability	Model Context Protocol bridge between CrewAI agents for structured tool calls	+5%

SECTION 2 — SYSTEM ARCHITECTURE

2. System Architecture

2.1 High-Level Architecture Overview

FinGuard AI is structured as a layered cloud-native system. The five layers — Presentation, API Gateway, AI/ML Engine, Data, and Infrastructure — are each independently deployable and communicate through well-defined contracts.



2.2 Data Flow — End to End

1. User uploads broker CSV via drag-and-drop interface or REST API.
2. FastAPI upload endpoint validates file, detects broker format (schema fingerprinting), and stores raw file in Azure Blob Storage with a job ID.
3. Azure Data Factory pipeline is triggered: raw CSV lands in the Bronze layer (Azure Data Lake Gen 2).
4. Databricks PySpark job cleans, normalises, and engineers features (50+ columns). Output is Delta tables in Silver and Gold layers.
5. Azure ML managed endpoint receives Gold-layer features and returns: risk label + probability, anomaly scores, behaviour cluster, and 30-day P&L forecast.
6. CrewAI orchestrator spawns all 4 agents in parallel. Agents call ML results, RAG queries, and external tools.
7. Agent outputs are assembled into a structured response JSON and stored in CosmosDB.
8. React frontend polls via WebSocket for real-time updates; Power BI refreshes on Azure's scheduled trigger.

SECTION 3 — TECHNOLOGY STACK

3. Full Technology Stack

Layer	Technology	Azure Service	Purpose
Frontend	React 18 + TypeScript + Vite + TailwindCSS + Recharts	Azure Static Web Apps	Responsive SPA UI
Backend API	Python 3.11 + FastAPI + Uvicorn + Pydantic v2	Azure App Service / Container Apps	REST APIs, auth, orchestration
ML Pipeline	scikit-learn, XGBoost, Isolation Forest, SHAP	Azure Machine Learning Studio	Training, tuning, inference
GenAI / Agents	LangChain + CrewAI + AutoGen + LlamaIndex	Azure OpenAI (GPT-4o)	Multi-agent orchestration
Vector DB	FAISS + ChromaDB + Azure AI Search	Azure Cognitive Search (semantic)	RAG retrieval, embeddings
Data Pipeline	PySpark + Pandas + Great Expectations	Azure Data Factory + Databricks + Fabric	ETL, validation, staging
Database	PostgreSQL (primary) + CosmosDB (NoSQL)	Azure SQL / Azure CosmosDB	Structured + document storage
Analytics	Power BI + DAX + Python visuals	Power BI Service (Embedded)	Real-time dashboards
Auth & Security	Azure AD B2C + OAuth 2.0 + JWT	Azure Key Vault + Azure AD	SSO, secrets management
DevOps / CI-CD	Docker + GitHub Actions + Pytest + Ruff	Azure Container Registry + ACR	Automated build, test, deploy

3.1 Why These Choices

FastAPI over Flask

FastAPI provides native async support, automatic OpenAPI/Swagger documentation, Pydantic v2 validation with zero boilerplate, and 3× better throughput than Flask for I/O-bound workloads. The type-safe request/response contracts eliminate an entire class of bugs during development.

XGBoost + Isolation Forest over Neural Networks

Tabular financial data with 5,000–50,000 rows is exactly where gradient-boosted trees dominate neural networks in both accuracy and training speed. Isolation Forest is the industry standard for unsupervised anomaly detection in transaction data. Both are natively supported in Azure ML Studio with MLflow tracking.

CrewAI + AutoGen over LangChain Agents

CrewAI provides explicit role-based agent definitions and structured task handoff, which makes the multi-agent workflow auditable — critical for a finance application. AutoGen handles the orchestration layer with built-in retry logic and conversation history management.

FAISS + Azure AI Search (Hybrid RAG)

Local FAISS provides microsecond similarity search during development and low-cost inference. Azure AI Search adds semantic ranking, hybrid BM25+vector search, and enterprise security. Using both gives the system a local fallback and cloud-scale capability.

SECTION 4 — MACHINE LEARNING PIPELINE

4. Machine Learning Models

FinGuard AI deploys five distinct ML models, each solving a specific analytical question about the user's trading behaviour. All models are trained in Azure Machine Learning Studio with MLflow experiment tracking, persisted as ONNX or pickle artifacts, and served via managed Azure ML endpoints.

Model	Algorithm	Features	Output	Serving	Explainability
Risk Classifier	XGBoost Classifier	Trade-level features: freq, avg_hold_time, sector_conc, volatility exposure	Conservative / Moderate / Aggressive + probability	Azure ML managed endpoint	SHAP waterfall for every prediction
Anomaly Detector	Isolation Forest + DBSCAN	Z-scored trade features: lot_size, time_of_day, rapid_reversal_flag	Anomaly score 0–100, contamination=0.05	Batch inference via ADF trigger	Top-3 feature contributions per alert
Loss Predictor	XGBoost Regressor	Holding period, entry price vs 52w high, sector momentum	Predicted P&L % for next 30 days	On-demand API call	Confidence interval ± SHAP
Behaviour Profiler	KMeans Clustering (k=5)	Panic score, revenge_trade_count, session_overtrade_ratio	Cluster label + psychological archetype name	Nightly batch job	Radar chart of 6 behavioural axes
Time-Series Forecast	Prophet + LSTM	Daily portfolio value, rolling 30-day window	7-day forward portfolio value forecast	Scheduled ADF pipeline	Trend + seasonality decomposition

4.1 Feature Engineering Pipeline

All features are computed from raw trade-level CSV data by the Databricks PySpark pipeline before being passed to any model. Key engineered features include:

Trade-Level Features

- ▶ holding_time_days: exit_date – entry_date (core for STCG vs LTCG boundary detection)
- ▶ rapid_reversal_flag: 1 if same stock bought and sold within 30 minutes (wash trade signal)
- ▶ lot_size_zscore: Z-score of quantity vs user's own historical distribution (detects unusual position sizing)
- ▶ session_overtrade_ratio: trades_today / rolling_90d_avg_trades_per_day
- ▶ loss_chasing_flag: 1 if a sell below cost_basis is followed by re-buy of same stock within 48h
- ▶ sector_concentration_hhi: Herfindahl-Hirschman Index of portfolio weights (1.0 = fully concentrated)
- ▶ panic_sell_score: composite of sharp price drop + large sell quantity + deviation from hold pattern

Portfolio-Level Features

- `sharpe_ratio_30d`: rolling Sharpe ratio using daily P&L as returns
- `drawdown_pct`: max drawdown from portfolio peak in the window
- `benchmark_alpha`: portfolio return – Nifty 50 return over same period

4.2 Model Training Workflow (Azure ML)

9. Data is loaded from Azure Data Lake Gold layer into Azure ML datasets.
10. Experiment is created: `finguard-risk-v1`, `finguard-anomaly-v1`, etc.
11. Hyperparameter tuning runs via HyperDrive (grid search for XGBoost, parameter sweep for Isolation Forest contamination factor).
12. Best run is registered as a versioned Model in Azure ML registry.
13. Model is deployed to a managed online endpoint with auto-scaling (1–5 instances).
14. FastAPI backend calls the endpoint via the Azure ML SDK v2 on each prediction request.
15. SHAP values are computed server-side and returned alongside predictions for explainability.

SHAP Explainability — Why Every Prediction is Transparent

Every risk score and anomaly flag includes a SHAP waterfall breakdown: the top-3 features that pushed the score up or down, each with a human-readable label. For example: 'Your Reliance Industries holding is 38% of your portfolio (`sector_concentration_hhi` = 0.21), which contributes +18 to your risk score.' This transforms ML black boxes into trusted advisor outputs.

SECTION 5 — MULTI-AGENT AI SYSTEM

5. Multi-Agent System Design

The agent layer is the differentiated core of FinGuard AI. Rather than a single general-purpose chatbot, the platform uses four specialised agents each trained on a precise sub-domain, orchestrated by an AutoGen supervisor. This mirrors how a real financial advisory firm operates: different analysts hand off findings to each other before a final recommendation reaches the client.

ID	Agent Name	Technology	Responsibilities	Framework
Agent 1	Portfolio Analyst	LangChain + GPT-4o	Analyses diversification, sector exposure, P&L patterns, benchmarks against Nifty 50. Returns structured JSON with SHAP-ranked risk factors and plain-English commentary.	CrewAI Task 1
Agent 2	Fraud & Behaviour Detective	AutoGen + Isolation Forest	Ingests trade-level ML scores. Identifies panic selling, overtrading, wash trades, round-trip trades. Generates timeline narrative with evidence chains.	CrewAI Task 2
Agent 3	Regulatory AI Assistant	LlamaIndex RAG + AI Search	Answers SEBI, Income Tax, LTCG/STCG, MF tax queries. Retrieves exact regulation chunks, cites section numbers, provides worked examples with real numbers.	CrewAI Task 3
Agent 4	Strategy & Rebalancing Advisor	GPT-4o + Tools	Synthesises outputs of Agents 1–3. Generates 3–5 prioritised, actionable recommendations. Suggests rebalancing trades with target allocations and rationale.	CrewAI Task 4 (Orchestrator)

5.1 Agent Communication Architecture

Step 1 — User Query Routing

When a user asks a question, the FastAPI `/api/v1/agent/query` endpoint sends it to the AutoGen Orchestrator Agent. The orchestrator uses a GPT-4o function-calling step to classify the query into: `[portfolio_analysis, fraud_review, regulatory_lookup, strategy_request, or combined]`. This classification is deterministic (not hallucinated) because the function schema constrains the output to these exact labels.

Step 2 — Parallel Agent Execution (CrewAI Tasks)

For combined queries (e.g., 'Why is my risk score so high and what should I do?'), the orchestrator spawns Tasks for Agents 1, 2, and 4 in parallel. Each agent receives a structured context object

containing the user's ML scores, portfolio summary, and any retrieved RAG chunks. CrewAI manages the task queue and collects outputs.

Step 3 — Agent Output Synthesis

The Strategy Agent (Agent 4) acts as the synthesiser. It receives the outputs of Agents 1–3 and produces a final JSON with exactly 3–5 numbered recommendations. Each recommendation includes: the issue, the evidence (with numbers), the specific action, and the expected impact.

Step 4 — MCP Bridge (Bonus)

An optional Model Context Protocol (MCP) server exposes the ML inference endpoints as structured tools. This allows AutoGen agents to call ML models with the same syntax as regular tool calls, enabling seamless interoperability between the agent framework and the ML serving layer. This earns the +5% bonus credit.

Example Agent Interaction — User Question: 'Why am I losing money?'

Orchestrator → routes to Agents 1, 2, 4 simultaneously. Agent 1 finds: IT sector overexposure at 36%, drawdown of 8.2% in April. Agent 2 finds: 3 panic sells detected, revenge trading pattern post-dip. Agent 4 synthesises: '1. Reduce IT to 20% by trimming TCS. 2. Set a stop-loss rule: never sell more than 20% of a position in one session. 3. Add FMCG or Pharma for defensive balance.' Each point is backed by exact numbers from ML outputs.

5.2 Regulatory AI Assistant — RAG System Design

Document Corpus

The RAG system is built on a curated corpus of 200+ documents covering:

- ▶ SEBI regulations: LODR, ICDR, Insider Trading Regulations, Circular archives 2020–2025
- ▶ Income Tax Act: Sections 111A (STCG), 112A (LTCG), 80C deductions, 43(5) speculative business rules
- ▶ AMFI guidelines: Mutual fund taxation, exit loads, SIP return computation
- ▶ NSE/BSE circulars: F&O margin rules, settlement calendar, corporate action impacts

Chunking Strategy

Documents are split into 512-token chunks with 128-token overlap using LlamaIndex's SentenceSplitter. Each chunk is enriched with metadata: document_title, section_number, effective_date, and document_type. This metadata is used to filter retrieval (e.g., only fetch SEBI docs for a SEBI question, only IT Act for a tax question).

Retrieval Pipeline

User query → Azure OpenAI text-embedding-3-large → top-10 semantic chunks from Azure AI Search (hybrid BM25+vector) → GPT-4o prompt with retrieved context → structured response with exact section citations and a worked numeric example.

Why This Assistant is Different from ChatGPT

- ▶ Answers are grounded in exact document chunks — no hallucinated regulation numbers.

-
- ▶ Every answer includes a worked example with real ₹ numbers based on the user's actual portfolio data.
 - ▶ The system explicitly says 'I don't know' if no relevant chunk is retrieved (retrieval confidence below threshold).
 - ▶ Queries about the user's own trades can be cross-referenced: 'Your Wipro profit of ₹8,200 held for 9 months falls under STCG. Tax = $8,200 \times 15\% = ₹1,230$.'

SECTION 6 — REST API SPECIFICATION

6. REST API Specification

All 12 core endpoints are documented below. The full OpenAPI/Swagger spec is auto-generated by FastAPI at /docs. All authenticated endpoints require a valid Azure AD B2C JWT in the Authorization: Bearer <token> header.

Method	Endpoint	Name	Description	Auth
POST	/api/v1/upload/csv	Upload trading CSV	File upload, broker detection, async parse trigger	auth required
GET	/api/v1/portfolio/summary	Portfolio KPIs	Returns investment, P&L, risk score, diversification score	auth required
GET	/api/v1/portfolio/holdings	All holdings with SHAP values	Stock list, weights, sector, AI risk commentary	auth required
POST	/api/v1/ml/predict/risk	Trader risk classification	Returns label (conservative/moderate/aggressive) + probability	auth required
POST	/api/v1/ml/predict/anomaly	Anomaly detection per trade	Returns anomaly scores, flags, reason codes	auth required
GET	/api/v1/ml/behavioural	Behavioural pattern analysis	Panic score, overtrade score, revenge trading flag, discipline	auth required
POST	/api/v1/fraud/analyze	Full fraud engine run	Runs all anomaly models, returns risk heatmap + timeline	auth required
GET	/api/v1/fraud/alerts	Active fraud alert list	All alerts with severity, timestamp, description, action	auth required
POST	/api/v1/agent/query	Route query to correct agent	Orchestrator picks Portfolio/Fraud/Regulatory/Strategy agent	auth required
POST	/api/v1/rag/search	RAG document search	SEBI/IT Act query → top-k chunks + cited answer	auth required
GET	/api/v1/reports/generate	Generate PDF/Excel report	Full report package, async job, returns download URL	auth required
GET	/api/v1/health	Service health check	All subsystem status (DB, ML, agents, OpenAI)	public

6.1 CSV Upload Flow (Detailed)

16. Client sends multipart/form-data POST to `/api/v1/upload/csv` with the file and optional broker hint.
17. FastAPI validates MIME type (must be `text/csv` or `application/vnd.ms-excel`) and size (max 25MB).
18. BrokerDetector class reads the first 3 rows and matches column signatures against known schemas for Zerodha, Groww, Upstox, and AngelOne. Returns `detected_broker` and `confidence_score`.
19. File is stored in Azure Blob Storage with a UUID filename. A Job record is created in Azure SQL with `status=PENDING`.
20. ADF pipeline trigger is fired asynchronously. The API returns 202 Accepted with `job_id` and a polling URL.
21. Client polls GET `/api/v1/upload/status/{job_id}` every 3 seconds. Status progresses: `PENDING` → `PARSING` → `FEATURE_ENGINEERING` → `ML_SCORING` → `AGENT_ANALYSIS` → `COMPLETE`.
22. On `COMPLETE`, the frontend receives a redirect to the dashboard with the new portfolio loaded.

SECTION 7 — AZURE CLOUD ARCHITECTURE

7. Azure Cloud Services

Azure Service	Role in FinGuard AI	Status
Azure App Service / Container Apps	Host FastAPI backend; blue-green deployment; auto-scaling rules	Required
Azure Static Web Apps	Host React frontend; global CDN; preview environments per PR	Required
Azure OpenAI Service (GPT-4o)	LLM backbone for all 4 agents; embeddings (text-embedding-3-large)	Required
Azure Machine Learning Studio	Model training, experiment tracking (MLflow), managed endpoints	Required
Azure AI Search (Cognitive Search)	Semantic search over SEBI PDFs; vector index for RAG retrieval	Required
Azure Data Factory	Scheduled pipeline: CSV ingestion → Databricks → Delta Lake	Required
Azure Databricks (PySpark)	Feature engineering, cleaning, aggregation, ML feature store	Required
Azure SQL Database	User accounts, portfolio snapshots, trade records, alert history	Required
Azure CosmosDB	Agent conversation history, RAG chunks, real-time session data	Required
Azure Key Vault	All secrets (OpenAI keys, DB passwords, JWT secret, API keys)	Required
Azure AD B2C	User authentication, OAuth 2.0, SSO, JWT token issuance	Required
Azure Monitor + App Insights	API latency, error rates, ML model drift alerts, agent traces	Required
Azure Container Registry	Docker image storage; GitHub Actions pushes to ACR on merge	Required
Azure Fabric (Lakehouse)	Gold layer storage, Power BI direct lake connection	Bonus
Azure Event Hub + Data Activator	Real-time trade stream ingestion; alert triggering on anomalies	Bonus

7.1 Azure Data Engineering Pipeline (Required Component)

Bronze Layer — Raw Ingestion (Azure Data Factory)

ADF pipeline 'FinGuard-CSV-Ingest' is triggered on Blob Storage event (new file arrival). It copies the raw CSV to Azure Data Lake Gen 2 container /bronze/uploads/{user_id}/{date}/. No transformation occurs here — the principle is raw data preservation.

Silver Layer — Cleaning and Normalisation (Databricks PySpark)

PySpark notebook 'clean_trades.py' executes on an auto-terminating cluster. Steps: (1) Schema enforcement per detected broker. (2) Null handling — missing prices filled via Yahoo Finance API fallback. (3) Date parsing with IST timezone normalisation. (4) Duplicate trade deduplication by (trade_id, timestamp) composite key. (5) Great Expectations data quality checks — failure blocks pipeline and alerts via Azure Monitor. Output: Delta tables in /silver/trades/.

Gold Layer — Feature Store (Databricks + Azure Fabric)

PySpark notebook 'feature_engineering.py' computes all 50+ features (see Section 4.1). Output is partitioned Delta tables in /gold/features/ and mirrored to Azure Fabric Lakehouse for Power BI DirectLake connection. This gives Power BI sub-second query response on the full feature dataset.

7.2 Security Architecture

- ▶ **All secrets (API keys, DB passwords, JWT signing key) live exclusively in Azure Key Vault. No secrets in code, .env files, or GitHub.**
- ▶ FastAPI uses DefaultAzureCredential — no credentials are passed in code. The Container App's managed identity has Key Vault Reader role.
- ▶ Azure AD B2C handles all user authentication. The frontend never handles passwords; it only receives JWT tokens.
- ▶ All inter-service communication within Azure (ADF → Databricks, FastAPI → Azure SQL) uses managed identities — no connection strings.
- ▶ CORS is locked to the Static Web App domain. API rate limiting: 60 requests/minute per user, 10 requests/minute per upload endpoint.

SECTION 8 — FRONTEND & USER EXPERIENCE

8. Frontend Architecture & UX Design

8.1 Application Pages

Dashboard (Main View)

The command-centre view. Five KPI cards (Total Invested, Current Value, P&L, Risk Score, Diversification Score), a portfolio performance chart vs Nifty 50 benchmark, sector allocation panel, AI insight strip, and three bottom panels: fraud alerts, behavioural analytics, and sector returns heatmap. All data streams via WebSocket — the dashboard updates live without a page refresh.

Portfolio Intelligence Page

Deep-dive into portfolio composition. Treemap of holdings by weight, candlestick-inspired P&L timeline, SHAP feature importance chart for the user's risk score, holding period analysis (how many positions are STCG vs LTCG eligible), and the full AI Portfolio Analyst report with expandable sections.

Fraud Detection Center

Timeline-based transaction view where each trade is plotted on a chronological axis, colour-coded by anomaly score (green = clean, amber = watch, red = flagged). Clicking a flagged trade opens a drawer with: the ML anomaly score, SHAP feature breakdown (which signals triggered the flag), the Fraud Detective agent's narrative explanation, and a 'Mark as Reviewed' action.

Regulatory AI Assistant

A full-screen conversational interface. Left panel shows the chat thread. Right panel shows the source documents used for the current response (highlighted relevant chunk, document title, section number, effective date). Below the chat input is a row of suggested queries based on the user's own data ('You made a LTCG-eligible gain on Dr Reddy's — ask me about LTCG tax.').

Reports Page

On-demand PDF and Excel report generation. Four report types: Portfolio Health Report, Fraud & Risk Report, Tax Computation Worksheet (STCG/LTCG breakdown ready for ITR), and AI Recommendations Summary. Reports are generated async; the user is notified via in-app notification when the download link is ready.

CSV Upload Page

Drag-and-drop zone with broker auto-detection animation. After broker is detected, a preview table shows the first 10 rows with column mapping overlaid. A confidence badge shows 'Zerodha detected (94% confidence)'. The user can override if detection is wrong. A processing progress bar with stage labels (Parsing → Engineering → ML Scoring → AI Analysis → Ready) keeps the user informed.

8.2 Component Architecture

-
- ▶ **UploadZone: handles drag-drop, file validation, broker detection polling, progress animation.**
 - ▶ PortfolioChart: Recharts LineChart with dual Y-axis (portfolio value + Nifty benchmark), anomaly zone shading, and click-to-drill tooltips.
 - ▶ RiskMeter: SVG arc gauge, 0–100 range, colour transitions green→amber→red with animated needle.
 - ▶ FraudTimeline: D3-powered chronological trade plot with anomaly score colour encoding and drawer interaction.
 - ▶ AIChat: WebSocket-connected chat with streaming token display, typing indicator, source citation drawer.
 - ▶ SectorHeatmap: 3×3 grid with CSS colour interpolation driven by sector MTD returns.
 - ▶ BehaviourRadar: Recharts RadarChart with 6 behavioural axes: Discipline, Patience, Risk Control, Diversification, Emotion Control, Strategy Adherence.
 - ▶ SHAPWaterfall: Custom SVG waterfall chart showing feature contributions to risk score.

8.3 State Management

Zustand stores manage four global state slices: portfolio (holdings, metrics, ML scores), fraud (alerts, timeline, reviewed IDs), agent (conversation history, loading states, source documents), and auth (user, JWT token, permissions). API calls use Axios with a request interceptor that attaches the JWT and a response interceptor that handles 401 token refresh.

SECTION 9 — POWER BI ANALYTICS

9. Power BI Dashboard Design

The Power BI report has 8 pages, published to Power BI Service and embedded in the React frontend via the Power BI Embedded API with a row-level security (RLS) token — each user only sees their own data.

#	Report Page	Key Visuals & DAX Measures
1	Portfolio Overview	Total Investment KPI, Current Value KPI, P&L %, Risk Score gauge, Diversification Score ring chart. DAX: TotalPL = SUM(current_value) - SUM(cost_basis)
2	Holdings Analysis	Treemap by sector weight, Table of holdings with conditional formatting, Holding period histogram (STCG vs LTCG boundary). DAX: HoldingDays = DATEDIFF(entry_date, TODAY(), DAY)
3	P&L Timeline	Area chart of daily portfolio value, Benchmark overlay line, Rolling 30d return measure. DAX: Rolling30dReturn = CALCULATE(...)
4	Fraud & Anomaly Report	Scatter plot of trades (X=time, Y=anomaly score), High-risk trade table, Anomaly score distribution histogram, Overtrade sessions heatmap by day-of-week
5	Behavioural Profile	Radar chart of 6 behaviour dimensions, Cluster assignment badge, Behaviour trend over last 6 months, Peer comparison (anonymised cluster averages)
6	Sector Exposure	Donut chart sector breakdown, Sector return vs Nifty sector index comparison table, Sector concentration risk score over time
7	Tax Computation	STCG trades table with tax estimate column, LTCG trades table, Total estimated tax payable card, Harvest opportunity suggestions
8	AI Recommendations	Agent output tiles (one card per recommendation), Risk reduction impact projections, Action completion tracker

SECTION 10 — DEVOPS & DEPLOYMENT

10. DevOps and CI/CD Pipeline

10.1 GitHub Actions Workflow

CI Pipeline (ci.yml) — Every Pull Request

- ▶ Checkout code, set up Python 3.11 and Node 20.
- ▶ Run Ruff linter and mypy type checker (backend). Run ESLint and TypeScript compiler (frontend).
- ▶ Run pytest with coverage: target $\geq 80\%$ overall, 100% on API route handlers.
- ▶ Build Docker images (backend + frontend) — fails if build errors.
- ▶ Run integration tests against a local docker-compose stack (FastAPI + PostgreSQL + mock Azure services).

CD Pipeline (cd.yml) — Main Branch Merge

- ▶ Build production Docker images tagged with git SHA and 'latest'.
- ▶ Push to Azure Container Registry.
- ▶ Deploy backend to Azure Container App (blue-green: traffic split 10%→50%→100% over 30 minutes with automatic rollback on 5xx rate spike).
- ▶ Deploy frontend to Azure Static Web Apps via the Azure/static-web-apps-deploy action.
- ▶ Run smoke tests: hit /api/v1/health, upload sample CSV, verify processing completes.
- ▶ Post deployment summary to GitHub PR comment with links to live environment.

10.2 Docker Setup

The project uses three Dockerfiles and one docker-compose.yml for local development:

- ▶ Dockerfile.backend: python:3.11-slim base, installs requirements, runs uvicorn with hot-reload in dev, gunicorn in production.
- ▶ Dockerfile.frontend: node:20-alpine build stage → nginx:alpine serve stage (multi-stage build, final image <50MB).
- ▶ docker-compose.yml: backend + frontend + PostgreSQL + Redis (for job queue) + localstack (mock Azure Blob) — full local stack with one command: docker compose up.

SECTION 11 — FOLDER STRUCTURE

11. Project Folder Structure

The repository is organised for clarity, separation of concerns, and evaluator navigation. Any evaluator should be able to find any component within 15 seconds of opening the repo.

Path / File	Purpose
<code>finguard-ai/</code>	Project root
├─ <code>backend/</code>	FastAPI Python backend
└─ <code>app/main.py</code>	Application entry point, lifespan, middleware
└─ <code>app/api/v1/</code>	All route modules (upload, portfolio, ml, fraud, agent, rag, reports)
└─ <code>app/agents/</code>	CrewAI agent definitions (portfolio, fraud, regulatory, strategy)
└─ <code>app/ml/</code>	Model training scripts, inference helpers, SHAP utilities
└─ <code>app/rag/</code>	LlamaIndex ingestion, chunking, FAISS index build, query pipeline
└─ <code>app/services/</code>	Business logic (csv_parser, risk_engine, fraud_engine, report_gen)
└─ <code>app/models/</code>	Pydantic schemas, SQLAlchemy ORM models
└─ <code>app/core/</code>	Config, security (JWT), Azure SDK clients, logging setup
└─ <code>tests/</code>	pytest unit + integration tests (>80% coverage target)
├─ <code>frontend/</code>	React + TypeScript SPA
└─ <code>src/pages/</code>	Dashboard, Portfolio, Fraud, Assistant, Reports, Settings
└─ <code>src/components/</code>	Charts (Recharts), Cards, AIChat, UploadZone, FraudTimeline
└─ <code>src/stores/</code>	Zustand global state (portfolio, alerts, agent, auth)
└─ <code>src/api/</code>	Axios API client with interceptors and retry logic
├─ <code>ml/</code>	Standalone ML notebooks and training scripts
└─ <code>notebooks/</code>	EDA, feature engineering, model selection, SHAP analysis
└─ <code>train_risk.py</code>	XGBoost risk classifier training pipeline
└─ <code>train_anomaly.py</code>	Isolation Forest + DBSCAN anomaly detector
└─ <code>evaluate.py</code>	Unified evaluation: accuracy, F1, AUC-ROC, SHAP summary
├─ <code>data/</code>	Data layer
└─ <code>raw/</code>	Uploaded CSVs (Zerodha, Groww, Upstox, AngelOne formats)
└─ <code>schemas/</code>	Expected column schemas per broker format
└─ <code>sebi_docs/</code>	SEBI circulars, IT Act sections (for RAG ingestion)
└─ <code>sample/</code>	Demo CSV files for evaluator testing
├─ <code>pipelines/</code>	Azure Data Factory + Databricks pipelines
└─ <code>adf/</code>	ADF pipeline JSON definitions

└─ databricks/	PySpark notebooks (clean, transform, feature_store)
─ infra/	Infrastructure as Code
└─ azure/	Bicep templates for all Azure resources
└─ docker/	Dockerfile.backend, Dockerfile.frontend, docker-compose.yml
└─ k8s/	Kubernetes manifests (optional advanced deployment)
─ .github/workflows/	GitHub Actions CI/CD pipelines
└─ ci.yml	Build, lint, test on every PR
└─ cd.yml	Build Docker → push ACR → deploy Container App on main merge
─ powerbi/	Power BI report files (.pbix) + Python visuals
└─ docs/	Architecture diagrams, API Swagger export, ADRs

SECTION 12 — IMPLEMENTATION ROADMAP

12. 14-Week Implementation Roadmap

Timeline	Phase	Deliverables
Week 1–2	Foundation	Project setup, Azure provisioning, FastAPI skeleton, React scaffold, Docker baseline, GitHub Actions CI
Week 3–4	Data Layer	CSV parser (all 4 brokers), ADF pipeline, Databricks feature engineering, Azure SQL schema, data validation
Week 5–6	ML Models	XGBoost risk classifier, Isolation Forest anomaly detector, SHAP integration, Azure ML experiment tracking
Week 7	RAG System	SEBI PDF ingestion, LlamaIndex chunking, FAISS index, Azure AI Search, regulatory query pipeline
Week 8–9	Multi-Agent	CrewAI agents (Portfolio, Fraud, Regulatory, Strategy), AutoGen orchestration, MCP bridge (bonus)
Week 10	Frontend	React dashboard, all chart components, AI chat UI, upload flow, fraud timeline, Power BI embed
Week 11	Integration	End-to-end flow testing, API integration, agent ↔ ML ↔ RAG wiring, error handling, logging
Week 12	Deployment	Docker production build, GitHub Actions CD, Azure Container App deploy, Key Vault secrets, smoke tests
Week 13	Power BI	8-page report, DAX measures, ML output tiles, publish to Power BI Service, embed in frontend
Week 14	Polish & Demo	Demo video, documentation, reflection note, evaluator sample data, final capstone submission

SECTION 13 — ADVANCED FEATURES & DIFFERENTIATORS

13. Advanced Features Beyond the Basic Requirement

13.1 Broker Auto-Detection Engine

Most capstones ask the user to select their broker manually. FinGuard AI uses schema fingerprinting — the first 3 rows of the CSV are analysed to match column names, date formats, and value distributions against known broker schemas. The system supports Zerodha (tradebook, P&L export), Groww (portfolio CSV), Upstox (trade history), and AngelOne (contract notes).

Confidence scoring uses a weighted Jaccard similarity between detected columns and each broker's schema template. If confidence < 80%, the user is shown a column mapping UI where they can manually correct the mapping. This is a real production pattern used by fintech data aggregators.

13.2 Tax Computation Worksheet

The platform computes a line-by-line STCG/LTCG calculation for every closed trade, ready to be used for ITR (Income Tax Return) filing. For each trade: cost of acquisition, fair market value where applicable (for grandfathered LTCG), sale consideration, days held, applicable tax rate (15% STCG / 10% LTCG above ₹1L), and estimated tax. This is a feature that no current student capstone in the fintech domain offers.

13.3 Peer Comparison (Anonymised Benchmarking)

Using the KMeans behaviour cluster assignments, users can opt into anonymised peer comparison: 'Users in your risk cluster (Moderate-Aggressive) average a 12.3% annual return. Your portfolio returned 16.4% — you are in the top 28% of your peer group.' This is computed entirely from synthetic/anonymised cluster statistics — no user data is shared.

13.4 Tax Loss Harvesting Suggestions

Agent 4 (Strategy Agent) includes a tax loss harvesting module. It identifies unrealised losses in positions that could be booked before March 31 to offset capital gains from profitable trades, reducing total tax liability. The recommendation includes: current unrealised P&L of the position, estimated tax saving if booked, whether the 30-day wash sale period is a concern.

13.5 Model Drift Detection (Azure Monitor)

Azure ML managed endpoints report prediction confidence scores to Azure Monitor. A Logic App triggers an alert if: (1) average confidence drops below 0.70 across 100 consecutive predictions (distribution shift), or (2) the anomaly rate exceeds 15% of all trades (model becoming too sensitive).

This alert triggers a retraining pipeline notification to the developer. This is a production MLOps pattern rarely seen in student projects.

SECTION 14 — QUALITY, TESTING & ERROR HANDLING

14. Quality Engineering

14.1 Error Handling Strategy

Every error in FinGuard AI is handled at three levels: input validation (Pydantic), service-level (try/except with structured logging), and API-level (global FastAPI exception handlers). No raw Python exceptions ever reach the client. All errors are returned as RFC 7807 Problem Detail objects with machine-readable error codes.

- ▶ **CSV Parse Errors:** Descriptive messages identifying the failing column and row. The user sees: 'Column "Trade Date" on row 14 contains "not_a_date" — expected format DD-MM-YYYY.'
- ▶ **ML Model Errors:** If the Azure ML endpoint is unavailable, the system falls back to a rule-based risk score (trade frequency × sector concentration). The user sees a yellow banner: 'Using simplified risk estimate — AI models are temporarily unavailable.'
- ▶ **Agent Timeouts:** Each agent has a 30-second timeout. If an agent times out, its partial output is still shown with a note: 'Strategy recommendations are taking longer than expected — check back in 60 seconds.'
- ▶ **RAG Empty Results:** If no chunk scores above the retrieval threshold (0.75 cosine similarity), the assistant responds: 'I could not find specific SEBI documentation on this query. Here is what I know from general finance knowledge — please verify with a tax advisor.'

14.2 Testing Coverage

- ▶ **Unit tests (pytest):** All service-layer functions, ML feature engineering functions, broker detection logic. Target: 85% line coverage.
- ▶ **API integration tests:** All 12 endpoints tested with valid and invalid inputs, auth and no-auth cases. Uses TestClient (FastAPI's built-in sync test client) against a test PostgreSQL database.
- ▶ **Agent tests:** Each agent is tested with mocked Azure OpenAI responses (using pytest-mock) to verify output schema compliance without making real API calls.
- ▶ **E2E tests (Playwright):** Upload a sample Zerodha CSV → verify dashboard loads with correct KPIs → run the AI assistant with a known query → verify response contains a tax calculation.

14.3 Logging and Observability

FastAPI uses Python's structlog library to emit structured JSON logs. Every request is tagged with: request_id, user_id (hashed), endpoint, duration_ms, status_code, and (for agent calls) agent_name and token_count. Azure Application Insights collects all logs, traces, and custom metrics. A pre-built App Insights dashboard shows: API latency P50/P95/P99, Azure OpenAI token usage and cost per day, ML endpoint inference times, and data pipeline success rates.

SECTION 15 — SAMPLE DATA & EVALUATION GUIDE

15. Evaluator Demo Guide

15.1 Sample CSV Files Provided

The /data/sample/ folder contains four pre-built CSV files, one per broker format, all using realistic but fictional data:

- ▶ `zerodha_tradebook_demo.csv` — 87 trades, 12 stocks, April 2024–March 2025. Includes 3 intentionally anomalous trades (rapid reversals) and 2 panic sell events for fraud detection testing.
- ▶ `groww_portfolio_demo.csv` — 24 holdings, mutual funds and equities. Designed to trigger the high IT-sector concentration warning.
- ▶ `upstox_trades_demo.csv` — 156 trades, F&O and equity mix. Triggers the overtrading behavioural pattern.
- ▶ `angelone_pnl_demo.csv` — P&L statement format. Tests the alternate column schema mapping.

15.2 Demo Script (5-Minute Walkthrough)

23. Open the app. Show the empty dashboard with the upload prompt (clean empty state with helpful CTA).
24. Drag `zerodha_tradebook_demo.csv` onto the upload zone. Show broker detection (Zerodha, 96% confidence). Show the progress bar moving through all stages.
25. Dashboard populates. Walk through KPI cards (Risk Score: 68, 3 fraud alerts badge on sidebar).
26. Click on a red-flagged trade in the Fraud section. Show the drawer: anomaly score 94, SHAP breakdown, agent narrative.
27. Open the Regulatory Assistant. Ask: 'How much tax will I pay on my Wipro gain?' Show the response with exact numbers, section 111A citation.
28. Open the Reports page. Generate the Tax Computation Worksheet. Show the PDF download with STCG/LTCG breakdown.
29. Show the embedded Power BI report, page 4 (Fraud & Anomaly). Demonstrate cross-filter: click a flagged trade on the scatter → detail table filters automatically.

APPENDIX — QUICK REFERENCE

Appendix A — Key Design Decisions (Architecture Decision Records)

ADR-001: FastAPI over Flask

Decision: Use FastAPI. Reason: Native async, automatic Swagger, Pydantic validation, and type safety. Flask requires additional libraries to match FastAPI's default feature set. Status: Accepted.

ADR-002: CrewAI over pure LangChain agents

Decision: Use CrewAI for agent role definitions, AutoGen for orchestration. Reason: CrewAI's role-based model makes agent responsibilities auditable — critical for a financial application where 'which agent said what' must be traceable. Status: Accepted.

ADR-003: FAISS + Azure AI Search dual retrieval

Decision: Run FAISS locally for development, Azure AI Search for production. Reason: Eliminates dev-prod parity issues. The LlamaIndex retriever abstraction allows swapping between them with one configuration change. Status: Accepted.

ADR-004: XGBoost over deep learning for tabular ML

Decision: XGBoost for all classification/regression tasks. Reason: Financial tabular data with <100,000 rows favours gradient boosted trees over deep learning. XGBoost with SHAP provides better explainability for end users. Status: Accepted.

Appendix B — Environment Variables Reference

Variable	Source	Notes
AZURE_OPENAI_API_KEY	Azure Key Vault → KV-finguard-prod	Never in code or .env committed to git
AZURE_OPENAI_ENDPOINT	Azure Key Vault	e.g. https://finguard.openai.azure.com/
AZURE_ML_ENDPOINT_URL	Azure Key Vault	Managed endpoint URL from Azure ML
AZURE_SQL_CONNECTION	Azure Key Vault	Postgres connection string
AZURE_COSMOS_CONNECTION	Azure Key Vault	CosmosDB primary connection string
AZURE_SEARCH_KEY	Azure Key Vault	Azure AI Search admin key
AZURE_STORAGE_ACCOUNT	App Settings	Storage account name (not secret)
JWT_SECRET_KEY	Azure Key Vault	512-bit random secret for JWT signing
ENVIRONMENT	App Settings	dev / staging / production

FinGuard AI

Built to evaluate at senior engineer level.

Left Shift Program 2026 · T5 Data & AI Track · Financial Risk & Advisory Platform